

Beam Balancing System Project

Synopsis/summary of project.....	2
Video Clip of System:.....	2
System overview.....	3
Labeled System Sketch:.....	3
Overview Photos:.....	3
Component Block Diagram.....	4
Design Description.....	5
Sensors:.....	5
Motors:.....	5
Microcontrollers:.....	7
Software:.....	8
Schematic:.....	9
Control System Design and Performance.....	9
Model assumptions:.....	10
Simulink Model:.....	11
Transfer Functions:.....	11
Simulink Gains:.....	11
Measured Performance:.....	13
Gains from demonstration and recorded run:.....	13
Summary of relevant parameters:.....	14
Comparison of Predicted and Experimented Results:.....	15
Appendix A.....	17
Spec Sheet: TOF400C VL53L1X.....	17
Spec Sheet: Micro Servo Motor SG90.....	19
Spec Sheet: Vibration Motor.....	20
Spec Sheet: Arduino MEGA 2560.....	21
Spec Sheet: Arduino UNO R4 with Wi-Fi.....	22
Software Code.....	23
Beam Distance PID - hosted on the MEGA 2560.....	23
Beam Angle only PID - Hosted on the UNO R4.....	28

Synopsis/summary of project

We have built a ball balancing seesaw, using two second order systems to control the position of the ball. The first system is a sliding mass placed under the beam that is used to control the angle and rotational velocity of the beam. This is accomplished with a rack and pinion gear system, actuated by a fixed angle servo motor with a range of motion of 0-180 degrees. The angle of the beam is read by a distance sensor we place under one side of the beam.

The second 2nd order control system denotes the position of the ball on the beam. Position of the ball is taken by 2 distance sensors placed on each side of the beam. The position of the ball is affected by the angle of the beam. The angle of the beam is supposed to stabilize as the ball approaches a set position on the beam. This set position can be defined in the user GUI or scheduled to rotate between set values.

Video Clip of System:

[Click Here](#)

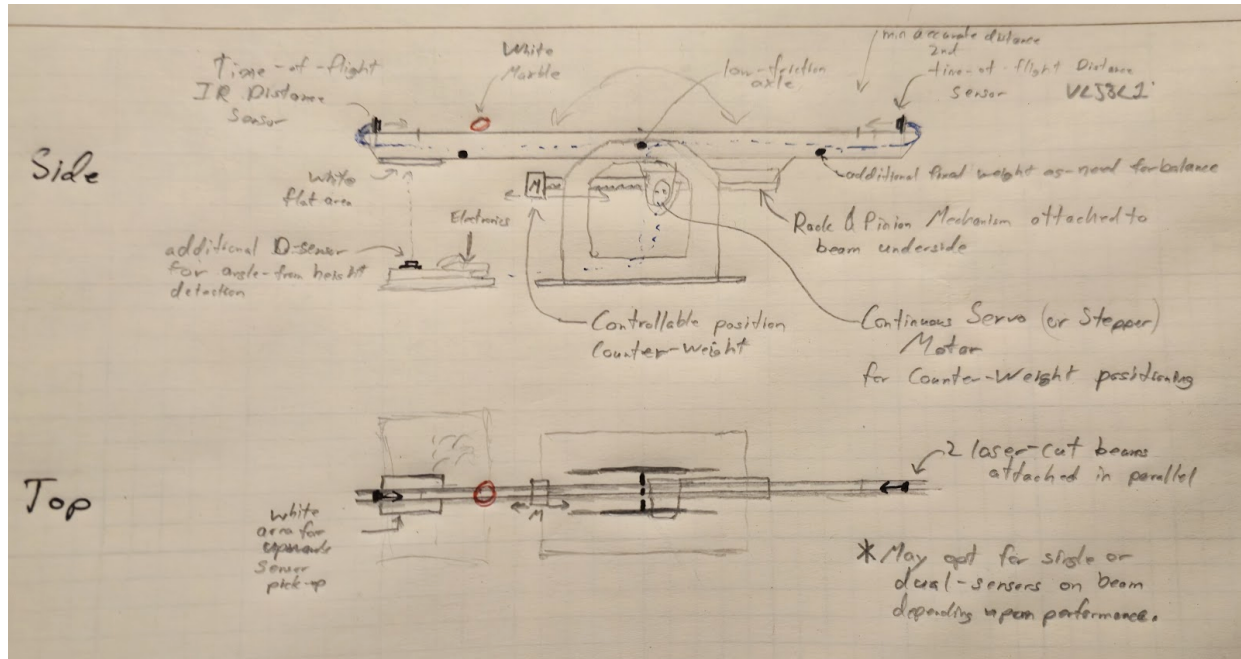
Here is a video from one of the runs from the system. This showcases the general performance of the system.

[Click Here](#)

This video is a clip of the system response when the ball is externally influenced and displaced from steady state.

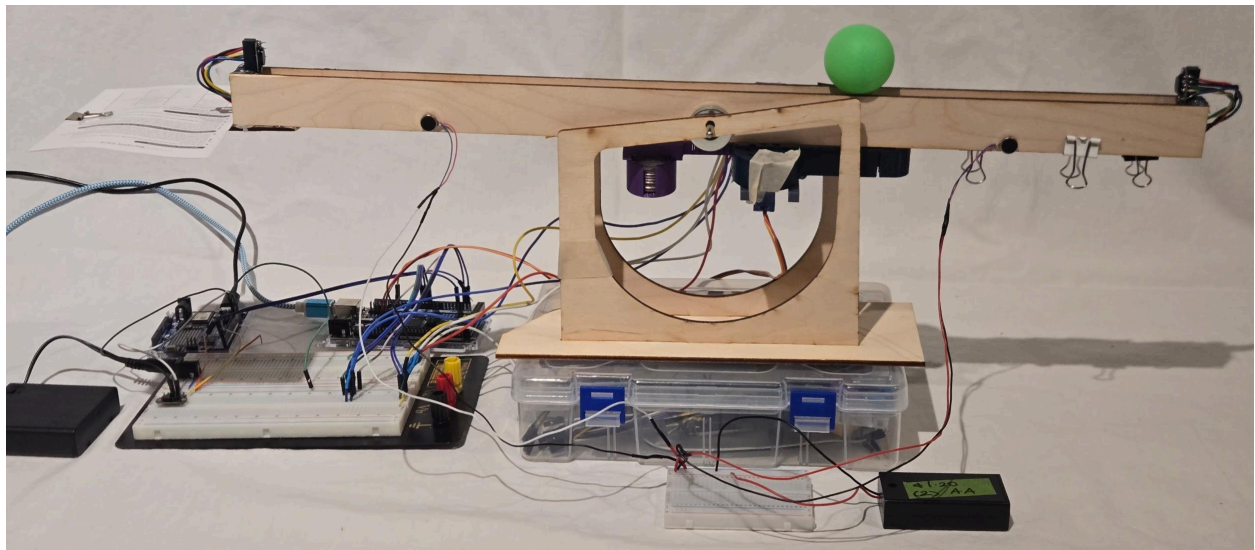
System overview

Labeled System Sketch:

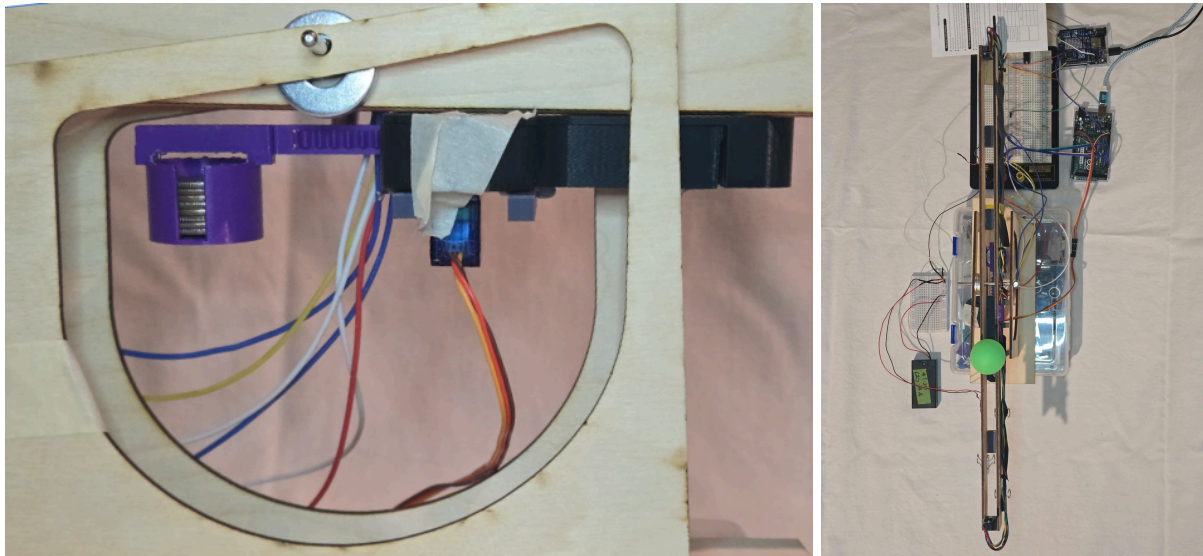


We have added vibration motors directly on the beam to reduce stationary friction. Additionally, one of the arches has had a redesign to accommodate the protrusion of the servo motor gear box.

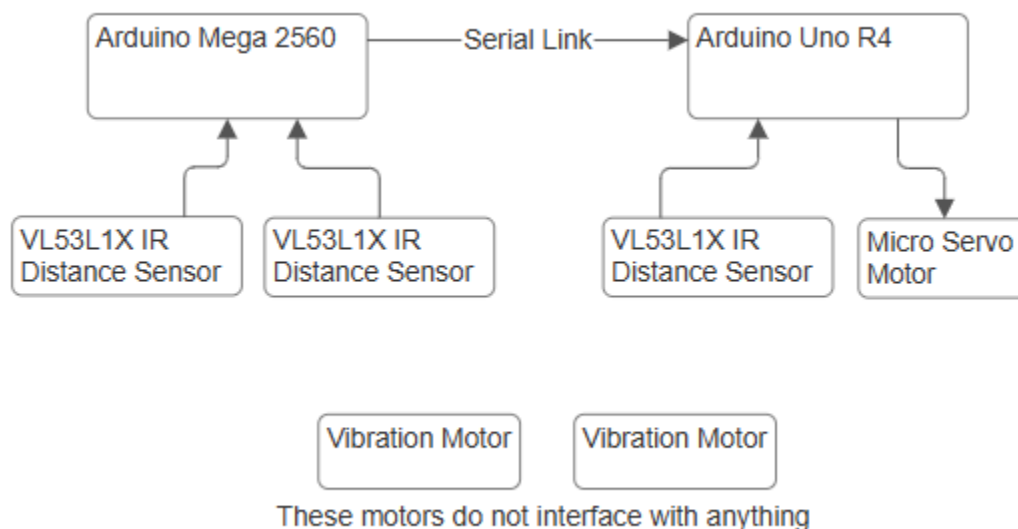
Overview Photos:



The image, same as in the sketch, there are 2 distance sensors on each end of the beam. A servomotor controlling a counter weight under the beam. And a 3rd distance sensor on the table, measuring the beam angle. Everything is connected to the micro controllers.



Component Block Diagram



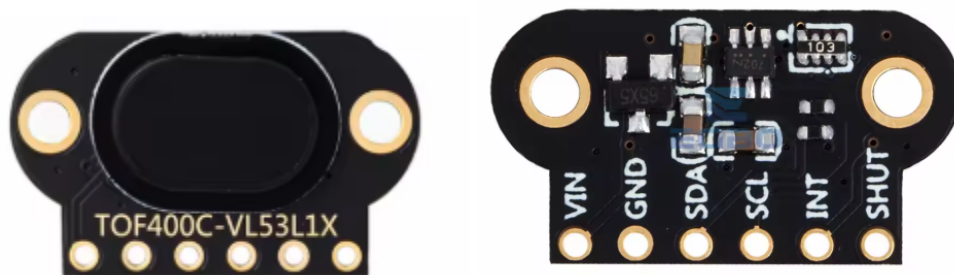
Design Description

Sensors:

The sensors we are using are three VL53L1X IR time-of flight distance sensors, used in detecting the angle of the beam and the position of the ball on the beam. These are small sensors mounted on a board with a black protector over where the sensor releases light and receives its signal.

For our purposes we calibrate the sensor by deciding where we would like the zero position of the item its sensing, and subtracting that value in order to make that desired zero position.

In terms of dynamics the general method TOF sensors work is by emitting modulated light and measuring the time it takes to reflect off an object and return to the sensor. It is calculated and outputs are transmitted to the SDA pin of the microcontroller.



[Spec Sheet](#) - See appendix

Motors:

For the primary actuation we are using 1 micro servo motor, SG90. This is a compact lightweight actuator used for precise motion control. It operates via PWM signals, allowing it to rotate 0-180 degrees and hold positions. Thus it provides an easy to use solution for accurate movements, which we utilized for this project.

Another reason we used this motor is for the calibration advantages. Once positions are initially set in correlation to the robot or system, it does not need re-calibration if reset. In our case we calibrated the rack and pinion to be fully collapsed when the servo was at zero position.

The dynamics of the micro servo are characterized by fast responsive movements with low inertia and high-torque holding capabilities. The operating angle is 180 degrees approximate, the stall torque is 1.2-1.8 kg approximately, no load speed is

0.12 sec/60 deg. The motor utilizes the PWM signal and internal gear reduction to convert high speed rotation to precise position, operating with a closed loop internal feedback system to reach the commanded position.



[Spec Sheet](#) - See appendix

The secondary actuator used in our build are small vibration motors. These motors are small circular motors, about 10 mm in diameter, that vibrate from side to side. They are DC powered eccentric rotating mass (ERM) actuators designed for haptic feedback like vibrations in mobile devices. We used these motors to prevent stick-tion (static friction) of the ball on the beam.

These actuators require no calibration for our usage, just need to be turned on and off, accomplished by connecting and disconnecting the battery.

The dynamics of this motor are relatively simple. As a ERM implies, it has a small internal weight offset from the center of rotation to generate a centripetal force, which is perceived as vibration when the magnetic motor spins rapidly. The vibration is dependent on the size of the mass and the offset from the center, it is supposed to rotate at 12000 RPM.



[Spec Sheet](#) - See appendix

Microcontrollers:

We are using 1 Arduino MEGA 2560, and 1 Arduino UNO R4 with Wi-Fi.

The Arduino MEGA 2560 is a robust microcontroller board design for complex projects requiring more I/O pins and memory. It has 54 digital pins (15 PWM included) , 16 analog inputs, and more. This provides extensive interfacing capabilities. We utilized this board because it could support multiple of our distance sensors with fast processing.

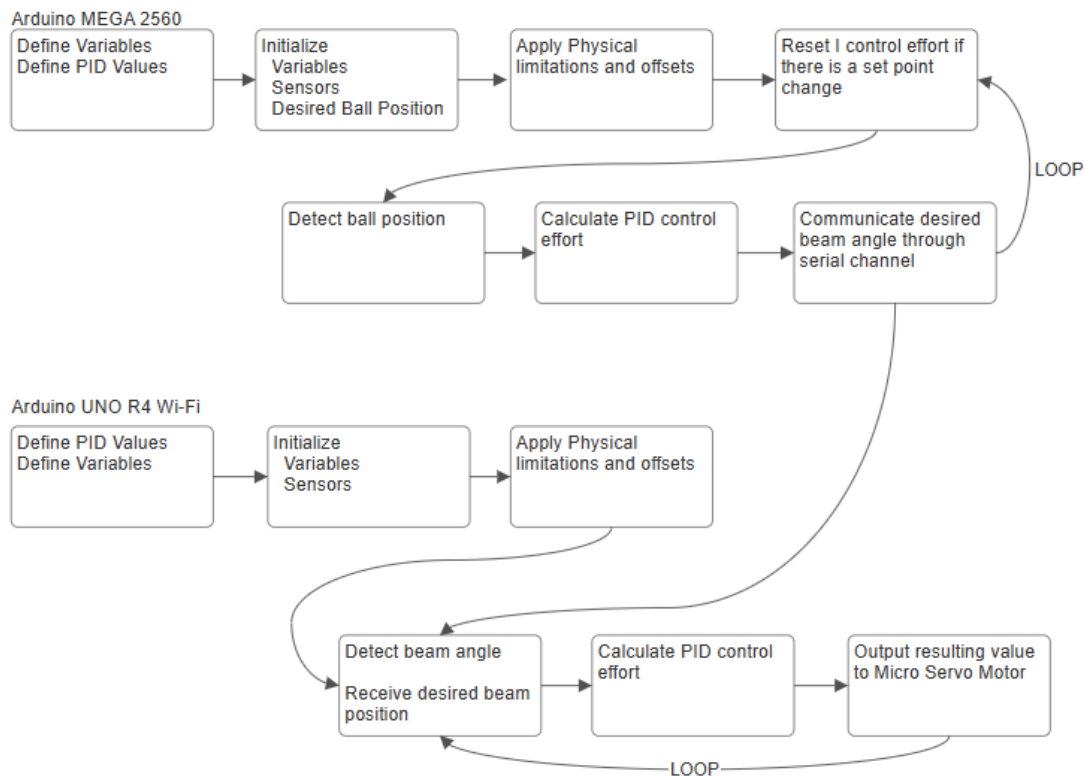
[Spec Sheet](#) - See appendix

The Arduino UNO R4 is a powerful 32-bit microcontroller board that modernizes the classic UNO form factor, featuring a Renesas ARM Cortex-M4 processor for significantly higher speed, memory, and performance. It retains 5V compatibility and the same pinout as the R3, ensuring compatibility with existing shields. The WiFi model adds built-in Bluetooth/Wi-Fi connectivity, a Qwiic connector, and an onboard LED matrix for advanced IoT projects. This board was utilized because we needed something to interface with the MEGA.

[Spec Sheet](#) - See appendix

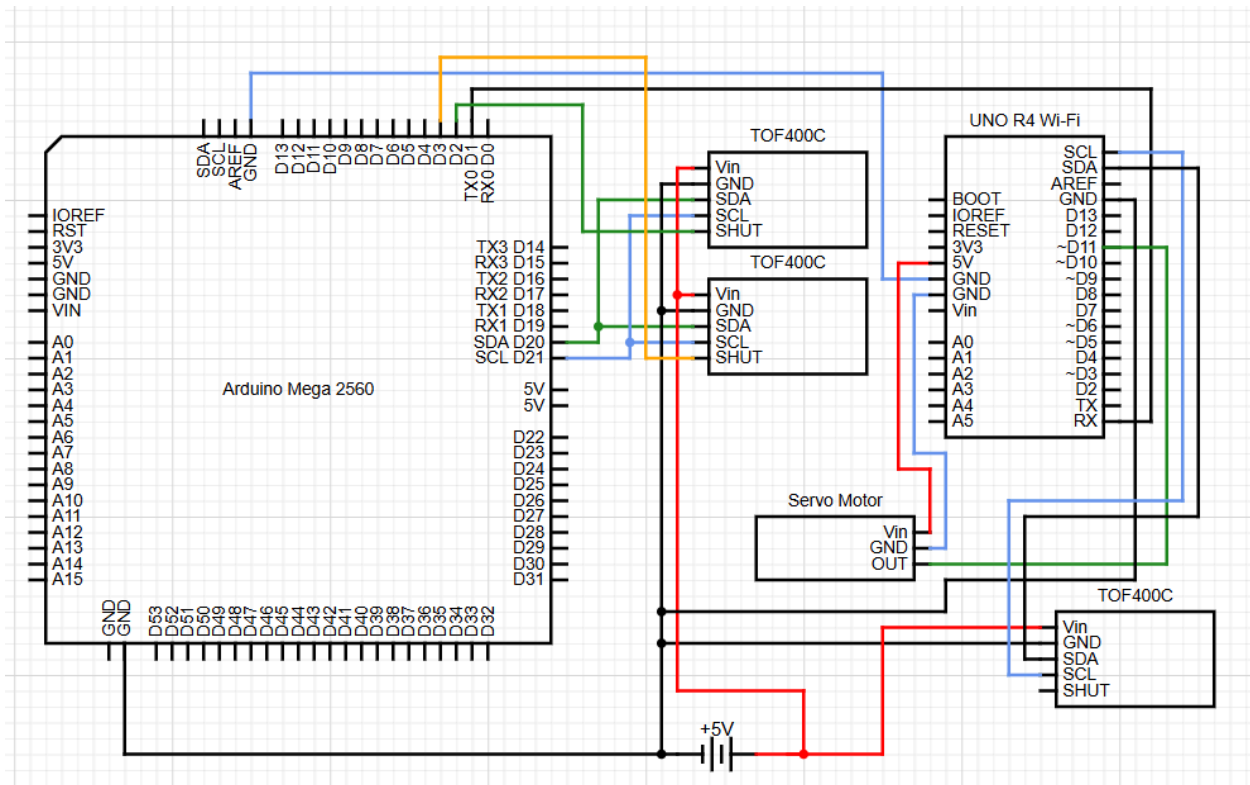
Software:

For software we programmed the two arduino microcontrollers to implement our control loops. The Arduino Mega handled our outer control loop of ball position as a function of beam angle. This desired angle value was then communicated to the Arduino UNO R4. The UNO R4 handled the inner control loop of motor position as a function of beam angle. Both of these control loops implemented individual PID controllers, and continuously iterated with real time position values.

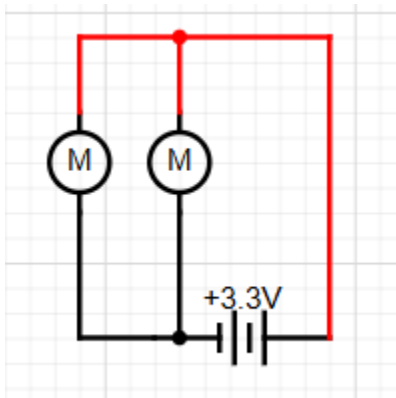


[Source Code](#) - See Appendix

Schematic:



For the vibration motors:



Control System Design and Performance

Our beam angle control transfer functions (but not angle measurement) made a number of assumptions, including the small angle approximation.

Model assumptions:

1. Small angle approximation for controlled counterweight torque.
2. Treatment of the combined servo-rack-and-pinion and counterweight system as a single object located at its center of mass, at a single distance below the beam.
3. Uniform density beam
4. The ball was assumed to be perfectly round, evenly balanced, and well approximated by a hollow mass shell.
5. Frictionless beam axle
6. Torques from wires were ignored, though we made significant efforts to minimize their spring-like forces, moving to a smaller number of more flexible wires from our original designs.
7. A 'drag' term linearly proportional to velocity, combining low speed air and rolling resistances (dominated by air for the pingpong ball). In our transfer function the magnitude of this term is controlled by coefficient "B".
8. Instant servo motor and sensor reaction and precision, and controller cycles. In reality sensor reaction was approx 25ms, controller cycle ~90ms (including sensor read), servo motor reaction was not measured but estimated at 10Hz.
9. Perfect sensor performance
10. Estimated limits on counterweight rate of movement and extremes of motion. (These limits were in response to simulated models attempting to move the counterweight impossibly far and fast)
11. Two vibration motors were added to counteract 'sticktion' of the ball. These were effective, but were not modeled for.

Variable	Value	Units
M, beam mass	180	g
m_w, mass of counterweight	53	g
m, mass of ball	2	g
L, beam length	55.6	cm
d, distance counterweight travels	5	cm
B, estimated linear drag coefficient	0.00075	kg/s

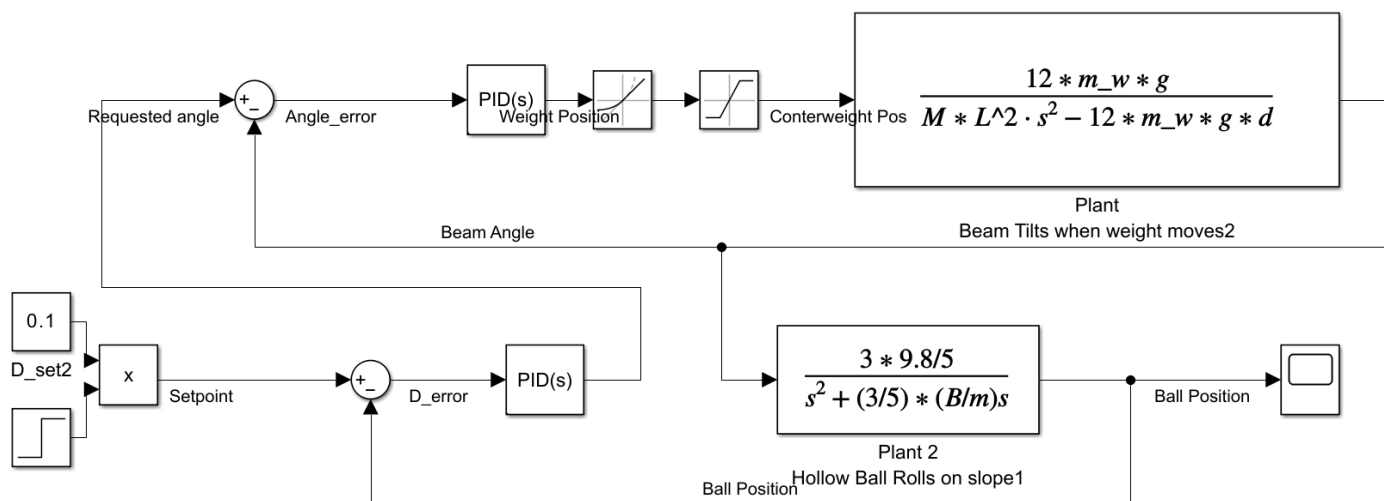
Counterweight Travel limits +/- 3.5 cm

Estimated Counterweight speed limit +/-1.5 m/s

Without the above limitations Simulink uses solutions which move the counterweight impossibly fast and beyond it's actual travel range. These bring the simulation more in line with realistic performance given our design limitations

Simulink Model:

We were fairly pleased that our simulink model (created prior to 4th order control) appeared to show similar oscillatory characteristics to our real system (if we included the counterweight movement limitation blocks).



Transfer Functions:

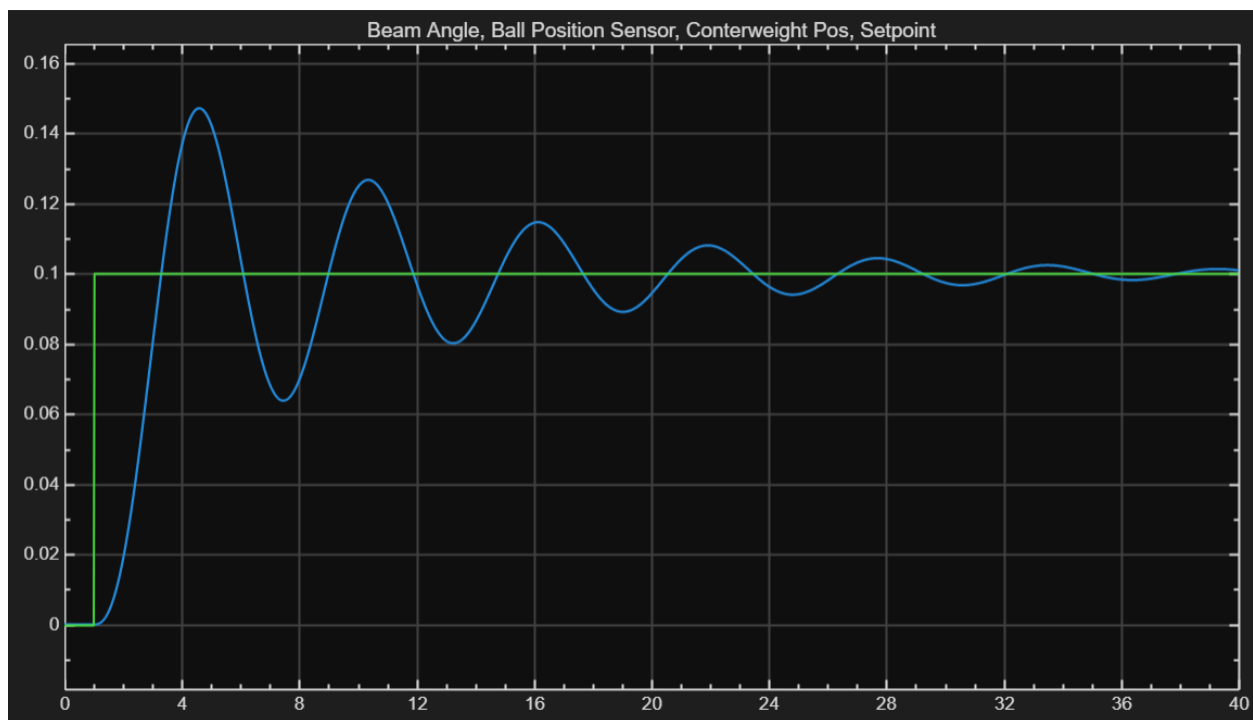
Based upon underlying physics of the system - See Block Diagram Above

Simulink Gains:

Variable	Value
"Inner" Beam Angle PID control Loop:	
K _p	20.0
K _i	0.05

K_d	3.0
"Outer" Ball Position PD control Loop:	
K_p	2.0
K_i	0.0
K_d	2.0

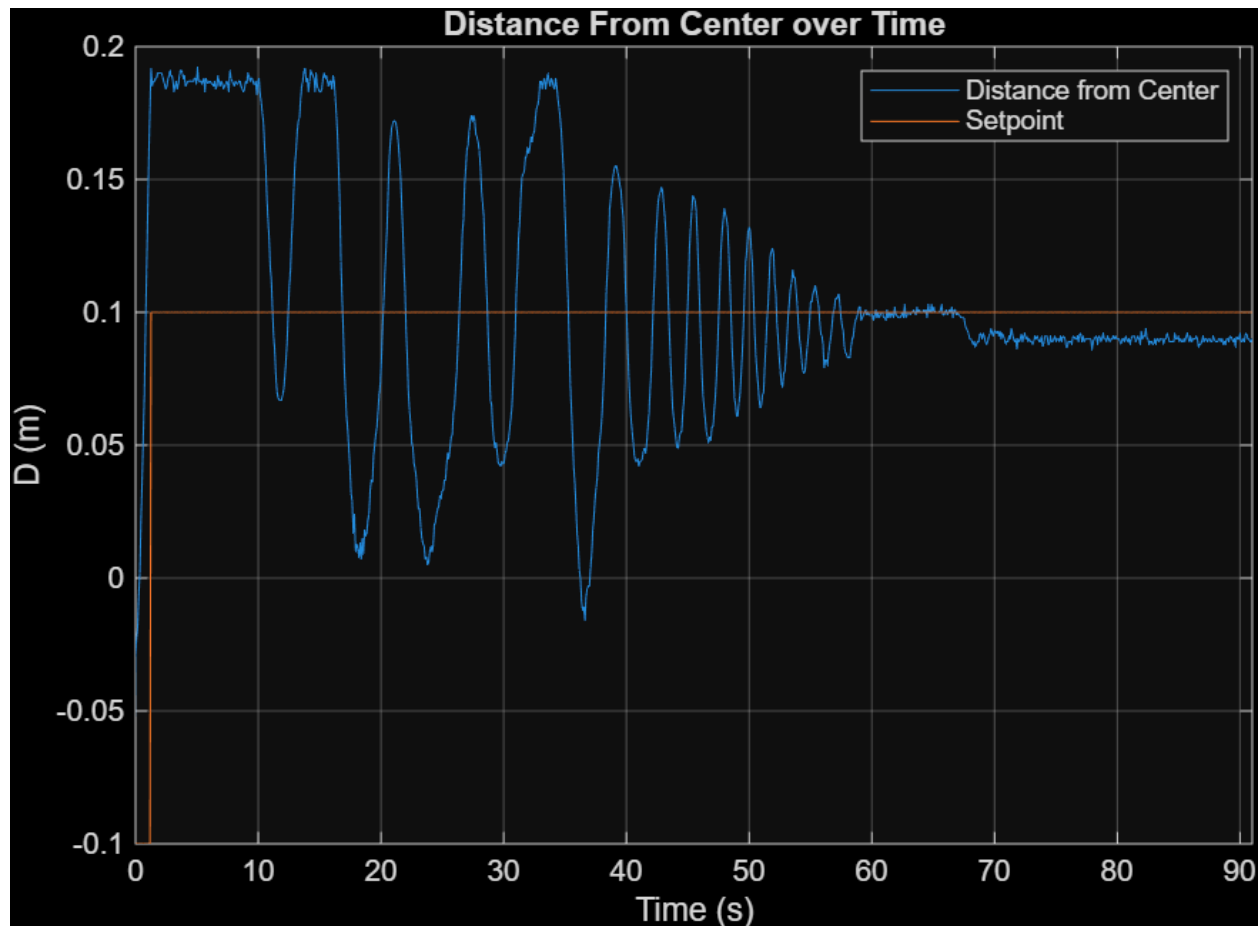
Note: the inner loop (both simulated and real, separately) were tuned to just beyond critically damped, with ~10% overshoot, in order to increase response time.



Graph is in units: meters for y-axis and seconds for x-axis

The simulink model of the system predicts highly oscillatory behavior, which settles around the set point after a period of time. This result is highly dependent on the type of ball we decide to use for the final system, as it greatly affects the physical attributes of the model. The model provided some starting gain values for the PID controller, but of course it required further tuning in physical application.

Measured Performance:



This example shows a period of non-linear 'bouncing' caused by the physical limitations of the beam, followed by a transition (at t=34s) to underdamped decaying oscillation and settling.

This data of the beam is in response to a step input, changing the desired ball position setpoint. This was our typical observer trial and matches the simulated mode. We did also perform some disturbance responses but decided they were not appropriate trials.

Gains from demonstration and recorded run:

Variable	Value
"Inner" Beam Angle PID control Loop:	
KA_p	5.5

KA_i	0.60
KA_d	1.925
“Outer” Ball Position PD control Loop:	
K_p	1.80
K_i	0.05
K_d	0.0

Summary of relevant parameters:

Parameter	Value	Performance Notes
Settling Time (ball position)	36 s	For <10cm changes or after non-linear bouncing settled.
Rise Time (ball position)	1.3 s	Overshoots significantly, period ~2.5s
Settling Time (beam angle)	1 s	Time for the beam to move and settle to a desired position.
Rise time (beam angle)	0.3 s	Relatively fast angle changes, exact time dependent on change magnitude. Tuned very slightly underdamped
Steady State Gain (beam angle)	Within sensor accuracy (± 0.2 degrees), for empty beam	Larger and highly dependent upon ball position when loaded

Our IR Time of flight sensors had a precision of approximately ± 2 mm when pointed at a flat surface (such as the card at the end we measured the distance upwards to for our beam tilt measurement). However, when pointed across the beam at a sphere, the accuracy was significantly lower (due to the curvature and variable

reflectivity of the ball most likely), typically $\pm 6\text{mm}$ in the more stable parts of the distance range, but up to $\pm 10\text{mm}$ further from the sensors or near the sensor transition zone.

The sensors also had a tendency to generate sporadic readings, most likely due to environmental interference (e.g. background, room lighting, nearby objects). The best understood source of this was a portion of our support structure we raised to give clearance to our servo motor mount. We adjusted the range of use of the two distance sensors to avoid that area, which helped significantly controlling on that portion of the beam, which previously had been unstable due to the poor distance measurement there.

Comparison of Predicted and Experimented Results:

Attempts were made to make the gains physically meaningful, but the inner control loop likely does not properly account for the translation of servo motor angle to counterweight net center of mass motion. The outer Ball position to requested angle control is thought to be in physically meaningful units, though our final values did differ significantly from the simulated ones needed for similar behavior.

Settling time was quite erratic, with frequent overshoots to the ends (linearity assumption broken), but in the domains where the system displayed more stable linear behavior the settling time was approximately 35 seconds, similar to the simulated settling time. However, disturbances or poorly modeled behavior (e.g. ball bouncing off ends of track), meant that in practice our settling time was often longer than this, and somewhat erratic.

Steady state errors were generally present, but typically not much more than one ball radius (20mm). Higher settled position accuracy could be obtained by letting integral control adjust but over even longer periods. But with significant risk of overshoot and re-oscillation so we elected not to operate in that mode very often.

By graphically comparing the behavior of the simulated model and real system we observe we are able to observe that for behavior after non-linear bouncing, both systems have similar settling approximations, 36 seconds approximate. The overshoot in that same range tended to be 0.06 m in the real system, which is comparable to the max overshoot of 0.05 m in the simulated system. It is noticeable that the real system has a raster time period/oscillation rate when graphically compared to the simulated system.

Overall in response to a step input the simulated system and real system have similar settling approximations. The amplitude and oscillations of the system are very

different between the real and simulated system. These differences can likely be attributed to the limited control cycle ($\sim 12\text{Hz}$) and actuation response times, as well as several factors such as an initial starting velocity in the real system the simulated does not account for, gain adjustments that had to be done to overcome factors assumed as insignificant for modeling, and the larger step input used in the real system to change the set point from -0.1 to 0.1 m while the model step input is from 0 to 0.1m . Overall the model does seem to approximate the behavior of the system.

Appendix A

Spec Sheet: TOF400C VL53L1X

Brand	ACEIRMC
Model Name	TOF400C
Connectivity Technology	I2C
Included Components	TOF400C VL53L1X 4M Laser Ranging Sensor Module
Operating System	Linux

About this item

- TOF400C VL53L1X 4M Laser Ranging Sensor Module TOF Time-of-Flight Distance IIC Output for Arduino Better Than TOF050C TOF200C
- TOF400C VL53L1X 4M Laser Ranging Sensor Module Operating Voltage: 3.0V-5V(DC)
- TOF400C VL53L1X 4M Laser Ranging Sensor Module Operating temperature: -20°C-70°C。 Operating current: 40mA (Max).
- TOF400C VL53L1X 4M Laser Ranging Sensor Module with Provide physical protection for the module, including preventing dust from entering.
- TOF400C VL53L1X 4M Laser Ranging Sensor Module Development routines/software: Arduino Demo / STM32 Demo

Key Specifications (TOF400C)

- **Sensor Chip:** STMicroelectronics VL53L1X (3rd Gen FlightSense)
- **Max Ranging Distance:** 4000 mm (4 meters)
- **Measurement Dead Zone:** 0 - 4 cm
- **Operating Voltage:** 3.0V - 5.5V DC
- **Communication Interface:** I2C (IIC) up to 400 kHz
- **Field of View (FOV):** 27°
- **Light Source:** 940 nm VCSEL (invisible, eye-safe)
- **Ranging Frequency:** Up to 50 Hz
- **Accuracy:** ±5% typical (1mm resolution)
- **Operating Temperature:** -20°C to 70°C 

Information specifically about the onboard light module below (not TOF400C board)

Features

Fully integrated miniature module

- 940 nm invisible laser emitter (Class 1)
- SPAD (single photon avalanche diode) receiving array with integrated lens
- Low power microcontroller running advanced digital firmware
- 4.9 x 2.5 x 1.56 mm

Fast, accurate distance ranging

- Fast and accurate long distance ranging
 - Up to 400 cm distance measurement
 - Up to 50 Hz ranging frequency
- Pin-to-pin compatible with the VL53L0X FlightSense ranging sensor
- 27° typical full field of view (FoV)
- Programmable region of interest (ROI) size on the receiving array allows reduction of the sensor FoV
- A programmable ROI position on the receiving array, provides multizone operation control from the host

Eye safety

- Class 1 laser device compliant with latest standard IEC 60825-1:2014 - 3rd edition

Easy integration

- Software driver and code example for turnkey ranging
- Single reflowable component
- Single power supply 2v8
- Can be hidden behind many cover glass materials
- I²C interface (up to 400 kHz)
- Shutdown and interrupt pins

2.1 Technical specification

Table 2. Technical specification

Feature	Detail
Package	Optical LGA12
Size	4.4 x 2.5 x 1.56 mm
Operating voltage	2.6 to 3.5 V
Operating temperature	-20 to 85°C
Receiver FoV (diagonal FoV)	Programmable from 15 to 27 degrees
Infrared emitter	940 nm
I ² C	Up to 400 kHz (fast mode) serial bus Programmable address default is 0x52

Spec Sheet: Micro Servo Motor SG90

Tiny and lightweight with high output power. Servo can rotate approximately 180 degrees (90 in each direction), and works just like the standard kinds but *smaller*. You can use any servo code, hardware or library to control these servos. Good for beginners who want to make stuff move without building a motor controller with feedback & gear box, especially since it will fit in small places. It comes with a 3 horns (arms) and hardware.

Specifications

- Weight: 9 g
- Dimension: 22.2 x 11.8 x 31 mm approx.
- Stall torque: 1.8 kgf·cm
- Operating speed: 0.1 s/60 degree
- Operating voltage: 4.8 V (~5V)
- Dead band width: 10 μ s
- Temperature range: 0 °C – 55 °C

Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is all the way to the left. ms pulse) is all the way to the right, ""-90" (~1ms pulse) is all the way to the left.

Spec Sheet: Vibration Motor

Brand	tatoko
Speed	12000 RPM
Voltage	3 Volts
Material	Stainless Steel
Item Weight	0.81 Ounces

About this item

- Battery Included: No; Rated Voltage: DC1.5- 3V
- Speed: 12000 RPM
- Body Size: 10 x 3mm/ 0.39" x 0.12" (D*T)
- Self adhesive Vibration Micro-motor for cell phone.
- Package included: 6 x DC Vibration Motors

Spec Sheet: Arduino MEGA 2560

Microcontroller	ATmega2560
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	54 (of which 15 provide PWM output)
Analog Input Pins	16
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	256 KB of which 8 KB used by bootloader
SRAM	8 KB
EEPROM	4 KB
Clock Speed	16 MHz
LED_BUILTIN	13
Length	101.52 mm
Width	53.3 mm
Weight	37 g

Spec Sheet: Arduino UNO R4 with Wi-Fi

Board	
Name	Arduino® UNO R4 WiFi
SKU	ABX00087 / ABX00087_CN / ABX00087_IN
Microcontroller	
Renesas RA4M1 (Arm® Cortex®-M4)	
Radio Module	
ESP32-S3-MINI-1-N8	
Pins	
Digital I/O Pins	14
Analog input pins	6
DAC	1
PWM pins	6
External interrupts	2,3
Communication	
UART	Yes, 1x
I2C	Yes, 1x
SPI	Yes, 1x
CAN	Yes 1x CAN Bus
Power	
Circuit operating voltage	5 V (ESP32-S3 is 3.3 V)
Input voltage (VIN)	6-24 V
DC Current per I/O Pin	8 mA

Software Code

Beam Distance PID - hosted on the MEGA 2560

```
#include <Wire.h>    // digital inputs, sensor read-in
#include <VL53L1X.h>  // distance sensor control
#include <Servo.h>    // servo control
#include <math.h>     // for trig

Servo myservo;
VL53L1X sensor1;
VL53L1X sensor2;

// Set Constants and create variables
const uint8_t XSHUT_1 = 2;
const uint8_t XSHUT_2 = 3;

const uint8_t ADDR_1 = 0x30;
const uint8_t ADDR_2 = 0x31;

const uint16_t PERIOD_MS = 25;

float D1;
float D2;
float D_c = 0;  // position from center
float Effort = 0;  // PID effort

float D_set = -90;

float K_p = 1.80; //1.55; //1.5//1.07;    //0.30;  //0.05-0.075    0.12,
0.09
float K_i = 0.05; //0.04; //0.05//0.02;    //Integral Gain
0.00175, 0.003
float K_d = 0.0; //0.0;    //0//0.6 ; ;    //5 ;
//Derivative Gain    0.2, 0.35    //Derivative Gain    0.2, 0.35

bool alternate_setpoint = true;  // switch between + and - setpoint every
t_switch
int t_switch = 90;
```



```

float Error = 0;
float int_error = 0;           // Integral error start value
float prev_error = 0;         //
float der_error = 0;
float prev_der_error = 0;
float last_D_set = D_set;     //remember last loop setpoint for
knowing when to reset I

float D_max = 400; // cap distances when object not found
float allowance = 0;
float r_ball = 25/2;
float D_left_offset = 214;//260 - r_ball ; //mm
float D_right_offset = 214;//260 - r_ball;

float servo_center = 71; //71
float servo_pos = 90;      // starting servo position
float MAX_SLEW = 90;

void setup() {
    Serial.begin(115200);
    // Serial.begin(230400);
    Serial1.begin(115200);

    // Wire.setClock(50000); // lower to 50000 if hard to read from longer
wires, default is 100000
    Wire.begin();

    myservo.attach(11);
    myservo.write(servo_center);

    // Shut both sensors down
    pinMode(XSHUT_1, OUTPUT); digitalWrite(XSHUT_1, LOW);
    pinMode(XSHUT_2, OUTPUT); digitalWrite(XSHUT_2, LOW);

    delay(10);

    // ---- Sensor 1 ---- Ball on Beam L

```

```

digitalWrite(XSHUT_1, HIGH);
delay(10);
sensor1.init();
sensor1.setAddress(ADDR_1);
sensor1.setDistanceMode(VL53L1X::Short);
sensor1.setMeasurementTimingBudget(1000*PERIOD_MS);
sensor1.setROISize(4, 4);
// sensor1.setROICenter(182);
sensor1.startContinuous(PERIOD_MS);

// ---- Sensor 2 ---- Ball on Beam R
digitalWrite(XSHUT_2, HIGH);
delay(10);
sensor2.init();
sensor2.setAddress(ADDR_2);
sensor2.setDistanceMode(VL53L1X::Short);
sensor2.setMeasurementTimingBudget(1000*PERIOD_MS);
sensor2.setROISize(4, 4);
sensor2.startContinuous(PERIOD_MS);

Serial.println(F("VL53L1X ready: D1, D2 (Short mode)"));
Serial.println(F("Format: D1:<mm>\tD2:<mm>\tD3:<mm>"));
}

```

```

void loop() {
    // timing - must be first
    static unsigned long t_last = 0;
    unsigned long t_now = micros();
    float dt = (t_now - t_last) / 1000000.0;
    if (dt > 0.5) dt = 0.5; // clamp first loop
    t_last = t_now;

    if (alternate_setpoint == true) {
        float t = millis() / 1000.0;
        float N = t_switch; // seconds per half-cycle
        if (fmod(t, 2*N) < N) {
            D_set = abs(D_set);
        } else {
            D_set = -abs(D_set);
        }
    }
}

```

```

    }
}

if (last_D_set != D_set){    // reset integral control when setpoint
changes
    int_error = 0;
    der_error = 0;
    prev_der_error = 0;
}

// Read sensor 1
sensor1.startContinuous(PERIOD_MS);
// delay(PERIOD_MS);
D1 = sensor1.read();
sensor1.stopContinuous();

// Read sensor 2 only
sensor2.startContinuous(PERIOD_MS);
delay(PERIOD_MS/2);
D2 = sensor2.read();
sensor2.stopContinuous();

D1 = min(D_max,D1);        // cap distances
D2 = min(D_max,D2);

if (D1 >= 500 && D2 >= 500) {
    D_c = D_set; // both sensors maxed out, no ball detected, stabilize
} else if (D1 > 135) {
    // D_c = D_right_offset - D2;
    D_c = D_right_offset - D2;
} else {
    D_c = D1 - D_left_offset;
}

// ball location PID loop
prev_error = Error; // save for next loop
Error = D_c - D_set;
// int_error = int_error + Error;        // update integral error

```

```

    int_error = constrain(int_error + Error * dt, -25000.0, 25000.0);
    // float der_error = (Error - prev_error);          // note: 10 frame
smoothing already done on T before error
    der_error = 0.7*((Error - prev_error)/dt) + 0.3*prev_der_error; //
adding some smoothing on D
    prev_der_error = der_error;

    float P = K_p * Error;
    float I = K_i * int_error;    // Integral 'effort'
    float D = K_d * der_error;    // Derivative 'effort'
    if (abs(Error) > allowance){
        Effort = P + I + D;
    } else {
        Effort = P;
        // int_error = 0;          // zero out the integral error    TODO, only if
we rest a while, not passing through!
    }          // drop D-control when close to setpoint, to avoid noise
kicks

    float theta_set = Effort/1000;
    Serial1.println(theta_set, 4); // 4 decimal places, in radians

    servo_pos = 45;
    myservo.write(servo_pos);

    // serial monitor
    Serial.print(F("D1:"));
    Serial.print(D1);
    Serial.print(F("\tD2:"));
    Serial.print(D2);

    Serial.print(F("\tD_c:"));
    Serial.print(D_c);
    Serial.print(F("\tD_set:"));
    Serial.print(D_set);
    Serial.print(F("\tP:"));
    Serial.print(P);
    Serial.print(F("\tI:"));

```

```

    Serial.print(I);
    Serial.print(F("\tD:"));
    Serial.print(D);
    Serial.print(F("\tE:"));
    Serial.print(Error);
    Serial.print(F("\tEffort:"));
    Serial.print(Effort);
    // Serial.println("");
    Serial.print(F("\tdt_ms:")); Serial.println(dt * 1000.0);
    t_last = t_now;
}

```

Beam Angle only PID - Hosted on the UNO R4

```

#include <Wire.h>    // digital inputs, sensor read-in
#include <VL53L1X.h> // distance sensor control
#include <Servo.h>   // servo control
#include <math.h>    // for trig

Servo myservo;
VL53L1X sensor3;

// Set Constants and create variables
const uint16_t PERIOD_MS = 20;
// float Dist3;

float servo_center = 71; //71
float servo_pos = 90;    // starting servo position
float MAX_SLEW = 90;

float theta;
float H0 = 185; //134+60; //112; // beam height (mm)
above sensor when at 0 degrees
float angle_sensor_arm = 290; //150; // how far out from the axis we are
measuring angle

float theta_set = 0 * PI/180;

```

```

float KA_p = 5.5;//4.5;//5;//0.75; //0.05-0.075    0.12, 0.09
float KA_i = 0.6;//0.3;//0.01;           //Integral Gain    0.00175,
0.003
float KA_d = 0.35*KA_p;///0.25//0.4*KA_p;//0.5 ;

float Error_A = 0;
float int_error_A = 0;           // Integral error start value
float prev_error_A = 0;         //
float prev_der_error_A = 0;

float D_max = 400;

void setup() {
    Serial.begin(115200);
    // Serial.begin(230400);
    while (!Serial); // wait for USB serial to connect
    delay(500);
    Serial.println("starting...");

    Serial1.begin(115200);

    myservo.attach(11);
    myservo.write(servo_center);

    // Wire.setClock(50000); // lower to 50000 if hard to read from longer
wires, default is 100000
    Wire.begin();

    // ---- Sensor 3 ---- Beam upwards
    // digitalWrite(XSHUT_3, HIGH);
    delay(10);
    // sensor3.init()
    if (!sensor3.init()) {
        Serial.println("sensor3 init failed!");
        while(1); // stop here so you can see the message
    };
    sensor3.setDistanceMode(VL53L1X::Short);
    sensor3.setMeasurementTimingBudget(1000*PERIOD_MS);
    sensor3.setROISize(8, 8);

```

```

    sensor3.startContinuous(PERIOD_MS);

    Serial.println(F("VL53L1X ready: Dist3, (Short mode)"));

}

void loop() {
    // timing - must be first
    static unsigned long t_last = 0;
    unsigned long t_now = micros();
    float dt = (t_now - t_last) / 1000000.0;
    if (dt > 0.5) dt = 0.5; // clamp first loop
    t_last = t_now;

    // Read sensor 3
    float Dist3 = sensor3.read(PERIOD_MS); // can drop careful time readout
    as this one doesn't interfere?
    if (sensor3.timeoutOccurred()) {
        Serial.println("sensor3 timeout!");
    }
    Dist3 = min(D_max, Dist3);

    // // Read theta_set from Mega
    if (Serial1.available()) {
        float received = Serial1.parseFloat();
        if (received > -1.0 && received < 1.0) { // sanity check
            theta_set = received;
        }
    }
    // theta_set = -0; // beam should hold level
    // theta_set = 5 * PI/180; // ~5 degrees, small positive tilt
    // theta_set = 15 * PI/180; // ~5 degrees opposite way

    float max_tilt = 15*PI/180;
    //if (abs(theta_set) > (max_tilt*PI/180)) {
    if (abs(theta_set) > (max_tilt)) {
        if (theta_set > 0) {

```

```

        theta_set = max_tilt;
    } else {
        theta_set = - max_tilt;
    }
}

// measure tilt angle theta from up-pointing dist sensor
float opposite = H0 - Dist3;
theta = -atan(opposite/angle_sensor_arm);

// angle PID loop
prev_error_A = Error_A; // save for next loop
Error_A = (theta - theta_set);
// int_error_A = int_error_A + Error_A; // update integral error
int_error_A = constrain(int_error_A + Error_A * dt, -250.0, 250.0);
// float der_error = (Error - prev_error); // note: 10 frame
smoothing already done on T before error
float der_error_A = 0.7*((Error_A - prev_error_A)/dt) +
0.3*prev_der_error_A;
prev_der_error_A = der_error_A;

float P_A = KA_p * Error_A;
float I_A = KA_i * int_error_A; // Integral 'effort'
float D_A = KA_d * der_error_A; // Derivative 'effort'
float Effort_A = (180/PI)*(P_A + I_A + D_A);

float target = servo_center + Effort_A;
servo_pos += constrain(target - servo_pos, -MAX_SLEW, MAX_SLEW);
servo_pos = constrain(servo_pos, 0, 180);
// servo_pos = 45;
myservo.write(servo_pos);
// myservo.write(servo_center+D_set/2);
// myservo.write(servo_center);

// serial monitor
Serial.print(F("\tDist3:"));
Serial.print(Dist3);
Serial.print(F("\ttheta:"));

```



```

Serial.print(theta*180/PI);

Serial.print(F("\ttheta_set:"));
Serial.print(theta_set*180/PI);
Serial.print(F("\tP_A:"));
Serial.print(P_A);
Serial.print(F("\tI_A:"));
Serial.print(I_A);
Serial.print(F("\tD_A:"));
Serial.print(D_A);
Serial.print(F("\tE_A:"));
Serial.print(Error_A);
Serial.print(F("\tEffort_A:"));
Serial.print(Effort_A);
// Serial.println("");
Serial.print(F("\tdt_ms:")); Serial.println(dt * 1000.0);
t_last = t_now;

}

```